TTI

Time To Interactive

The time at which a page becomes interactive (events wired up, etc).

Total Blocking Time (TBT)

Nov 7, 2019 • Updated May 4, 2020

Appears in: Metrics



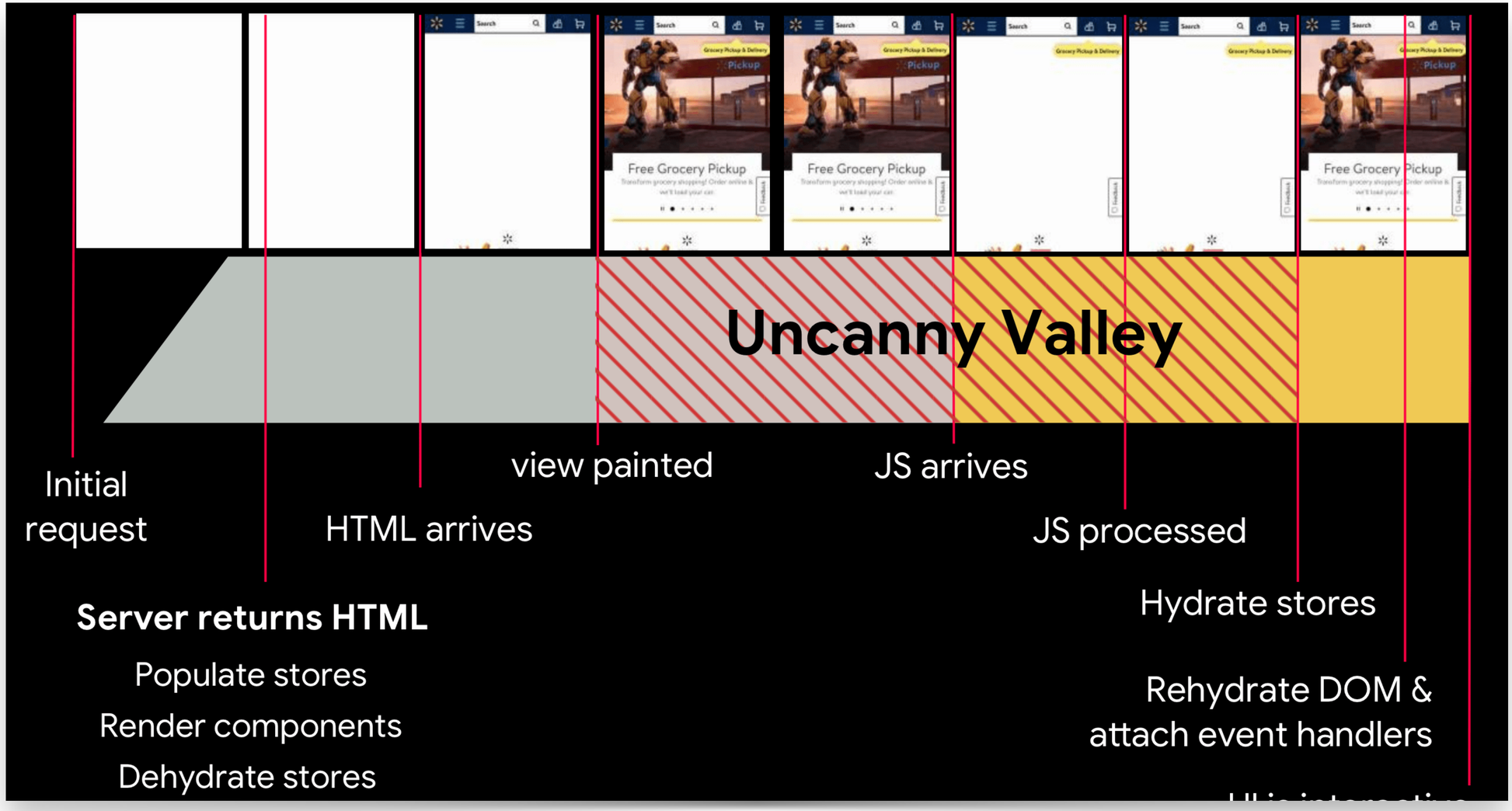
Philip Walton
Twitter · GitHub · Blog

★ Total Blocking Time (TBT) is an important, user-centric metric for measuring load responsiveness because it helps quantify the severity of how non-interactive a page is prior to it becoming reliably interactive—a low TBT helps ensure that the page is usable.

SHARE
SUBSCRIBE

What is TBT?

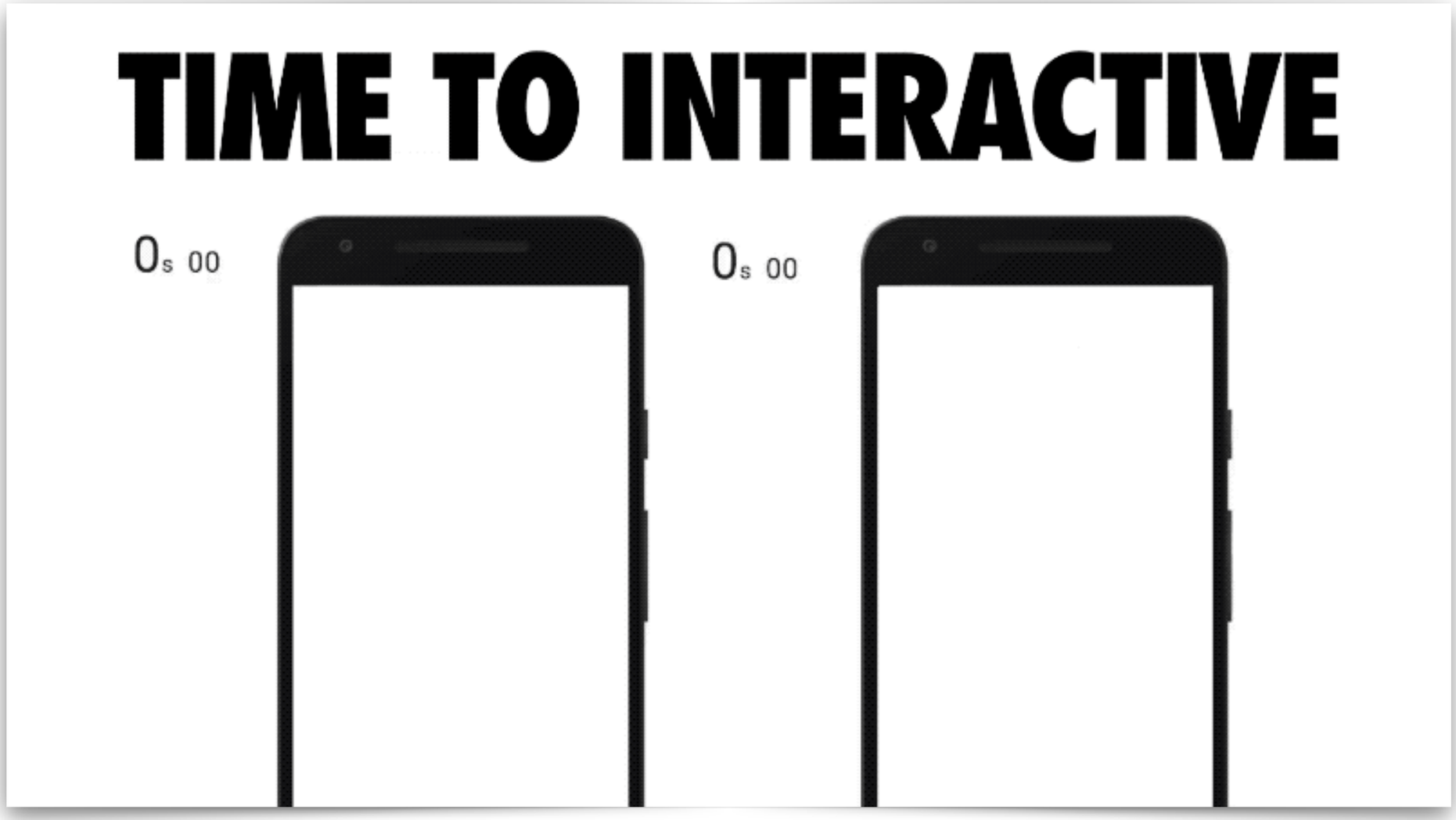
https://web.dev/tbt/



https://addyosmani.com/blog/rehydration/



https://twitter.com/NetflixUIE/status/923374215041912833



https://medium.com/@addyosmani/the-cost-of-javascript-in-2018-7d8950fbb5d4

“

we feel your baseline should be getting interactive in under 5 seconds on a slow 3G connection on a median mobile device.

Addy Osmani



https://medium.com/@addyosmani/the-cost-of-javascript-in-2018-7d8950fbb5d4

The Vanilla JS Toolkit

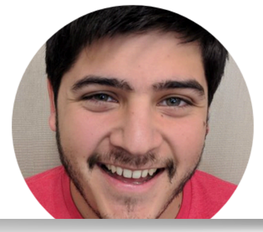
Methods & APIs Helpers Plugins Boilerplates Polyfills Learning

The Vanilla JavaScript Toolkit

A collection of JavaScript methods, helper functions, plugins, boilerplates, polyfills, and learning resources.

Vanilla JS *is a term for coding with native JavaScript features and browser APIs instead of frameworks and libraries.*

⏰ ***Last chance to join!*** *A new session of the Vanilla JS Academy starts on Monday. Register today and save 30%. Click here to learn more.*



Can you actually create a website without JS frameworks? Yes! Here's a collection of resources to get started.

- Ed Rivas

https://vanillajstoolkit.com/reference/

Reference Guide

A quick reference for commonly used JavaScript methods and browser APIs.

Unless otherwise noted, these work in all modern browsers, and IE9+. You can extend support back further with polyfills.

⏰ ***Last chance to join!*** *A new session of the Vanilla JS Academy starts on Monday. Register today and save 30%. Click here to learn more.*

On this page

- Selectors
- Loops
- Classes
- Styles
- Attributes
- Event Listeners
- Strings

filament group

What we do | Who we are | Our work | Articles | Code

Styling a Select Like It's 2019

Posted by **Scott** 12/18/2018

> **Update 12/19** The select now has totally consistent appearance in Internet Explorer 11 and 10, thanks to **a nice tip** from **Jelmer de Maat**

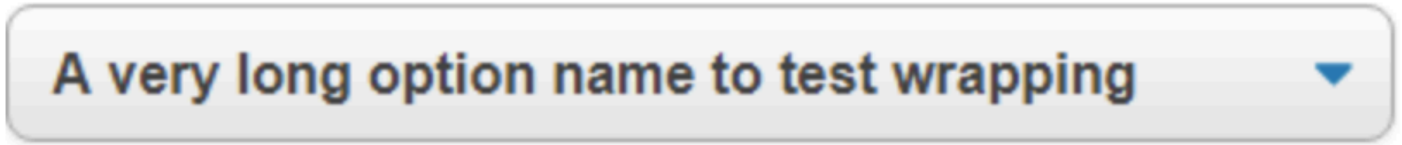
The `select` element has long been difficult to style consistently across browsers. To avoid its shortcomings in the past, we have used workarounds like styling a parent element, adding pseudo-elements, and even using JavaScript to construct a select-like control out of different elements that are easier to style. But workarounds are hard to maintain and use, not to mention the accessibility challenges that custom elements bring.

Recently, we'd seen some articles suggest that things haven't changed a great deal with `select`'s styling limitations, but I decided to return to the problem and tinker

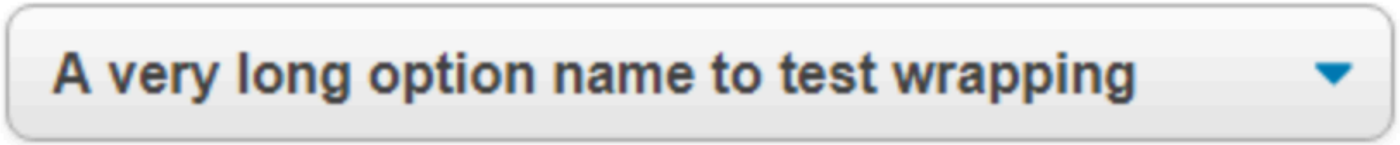


https://www.filamentgroup.com/lab/select-css.html

- Firefox



- Chrome



- Safari



https://www.filamentgroup.com/lab/select-css.html

Basic Snapper example

A snapper carousel with some thumbnail links.

Thumbnails are just regular links to a slide's ID attribute. The scrollbar is cropped from sight using the optional **`snapper_pane_crop`** div (only recommended when thumbnails or next/prev navigation is in play).



Installation
Quick start
The Why
Tree-Shaking
Compatibility
» **Command Line Interface**
Configuration Files
Differences to the JavaScript API
Loading a configuration from a Node package
Using untranspiled config files
Command line flags
Reading a file from stdin
» **JavaScript API**
rollup.rollup
rollup.watch
» **ES Module Syntax**
Importing
Exporting
How bindings work
» **Tutorial**

rollup.js

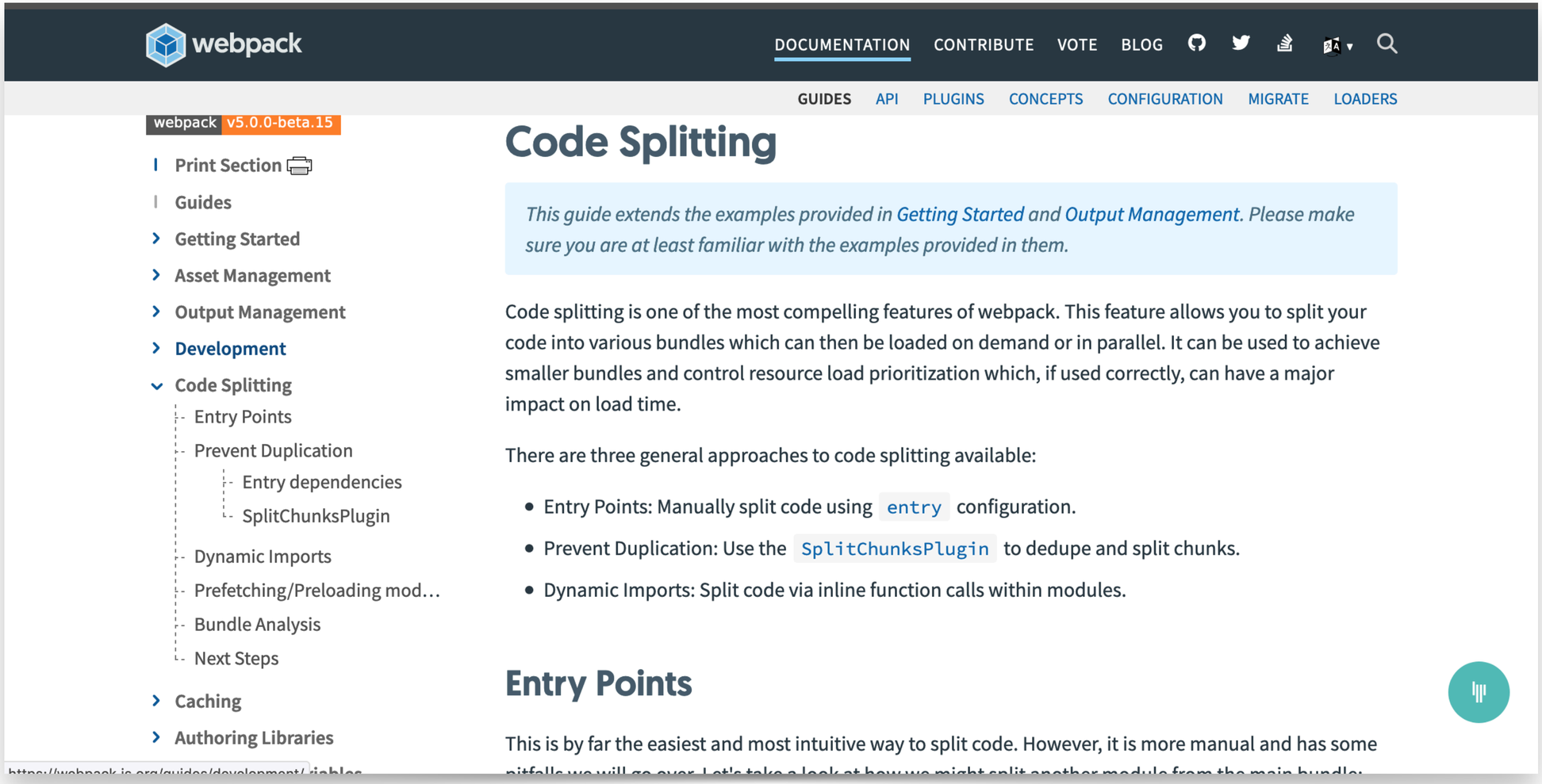
Introduction

Overview

Rollup is a module bundler for JavaScript which compiles small pieces of code into something larger and more complex, such as a library or application. It uses the new standardized format for code modules included in the ES6 revision of JavaScript, instead of previous idiosyncratic solutions such as CommonJS and AMD. ES modules let you freely and seamlessly combine the most useful individual functions from your favorite libraries. This will eventually be possible natively everywhere, but Rollup lets you do it today.

Installation

https://rollupjs.org/guide/en/#



https://webpack.js.org/guides/code-splitting/

Was $60.00 Now $50.00

105 Reviews / Write a Review

3.4 / 5

66% Recommend this item

Overall Fit:

RUNS SMALL | TRUE TO SIZE | RUNS LARGE

Filter By

Body Type | Height

Sort By

Highest Rated

JWall

Atlanta

Overall Fit:

★★★★★

Yes, I recommend this product.

Great fit and good value

June 26, 2018

Good T-shirts can be hard to

Home
Open DevTools
CSS
Console
Network
Storage
Command Menu
Mobile Simulation
DOM
JavaScript
Performance
Get Started
Optimize Website Speed
Overview
Performance Analysis Reference
Timeline Event Reference
Speed Up JavaScript Execution
How to Use the Timeline Tool
Diagnose Forced

Speed Up JavaScript Execution

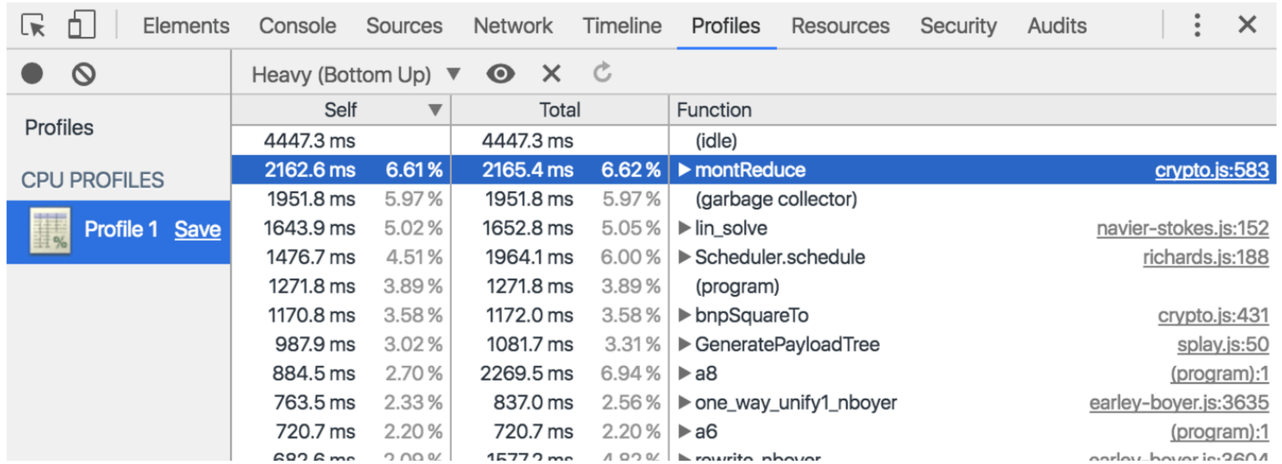


By Kayce Basques
Technical Writer, Chrome DevTools & Lighthouse



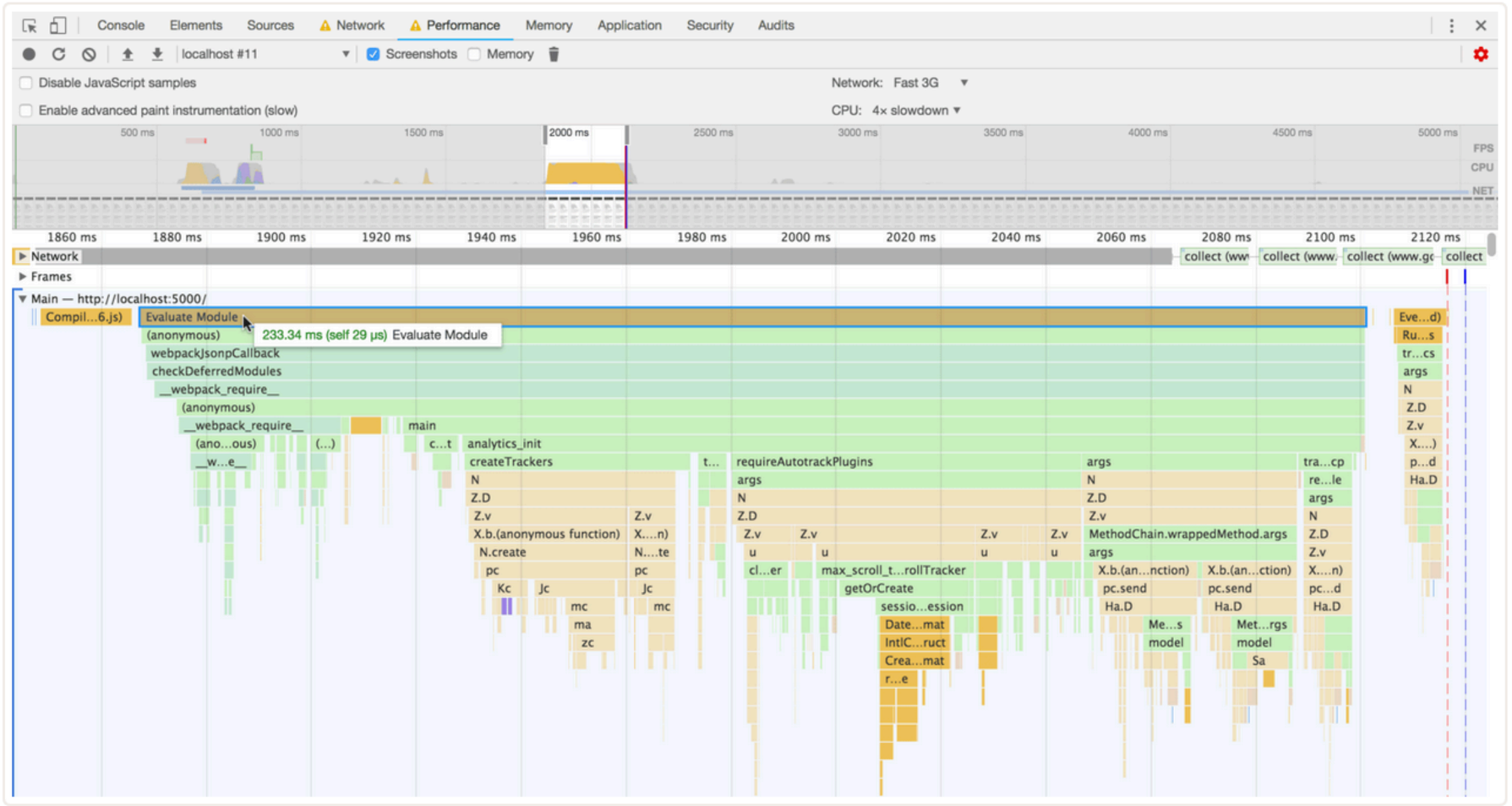
By Meggin Kearney
Meggin is a Tech Writer

Identify expensive functions using the Chrome DevTools CPU Profiler.



Contents

Record a CPU profile
View CPU profile
Change sort order
Exclude functions
View CPU profile as Flame Chart
Zoom in on specific parts of recording
View function details
Feedback



A performance trace of my site's JavaScript while loading (with network/CPU throttling enabled).

https://philipwalton.com/articles/idle-until-urgent/

First Input Delay (FID)

Nov 7, 2019 • Updated May 4, 2020

Appears in: Metrics



Philip Walton
Twitter · GitHub · Blog

★ First Input Delay (FID) is an important, user-centric metric for measuring load responsiveness because it quantifies the experience users feel when trying to interact with unresponsive pages—a low FID helps ensure that the page is usable.

We all know how important it is to make a good first impression. It's important when meeting new people, and it's also important when building experiences on the web

SHARE
SUBSCRIBE

https://web.dev/fid/

MDN web docs
moz://a

Technologies ▼ References & Guides ▼ Feedback ▼ Search MDN Sign in

window.requestIdleCallback()

Web technology for developers > Web APIs > Window > window.requestIdleCallback()

English ▼

On this Page

- Syntax
- Example
- Specifications
- Browser compatibility
- See also

Related Topics

Window

▼ Properties

- applicationCache
- caches
- closed

This is an experimental technology
Check the Browser compatibility table carefully before using this in production.

The **`window.requestIdleCallback()`** method queues a function to be called during a browser's idle periods. This enables developers to perform background and low priority work on the main event loop, without impacting latency-critical events such as animation and input response. Functions are generally called in first-in-first-out order; however, callbacks which have a `timeout` specified may be called out-of-order if necessary in order to run them before the timeout elapses.

You can call `requestIdleCallback()` within an idle callback function to schedule another callback to take place no sooner than the next pass through the event loop.

A `timeout` option is strongly recommended for required work, as otherwise it's possible multiple seconds will elapse before the callback is fired.

https://developer.mozilla.org/en-US/docs/Web/API/Window/requestIdleCallback

Window.requestAnimationFrame()

On this Page

Syntax
Example
Notes
Specification
Browser compatibility
See also

Related Topics

Window

▼ Properties

applicationCache
caches

The `window.requestAnimationFrame()` method tells the browser that you wish to perform an animation and requests that the browser calls a specified function to update an animation before the next repaint. The method takes a callback as an argument to be invoked before the repaint.

> **Note:** Your callback routine must itself call `requestAnimationFrame()` if you want to animate another frame at the next repaint.

You should call this method whenever you're ready to update your animation onscreen. This will request that your animation function be called before the browser performs the next repaint. The number of callbacks is usually 60 times per second, but will generally match the display refresh rate in most web browsers as per W3C recommendation. `requestAnimationFrame()` calls are paused in most browsers when running in background tabs or hidden `<iframe>`s in order to improve performance and battery life.

The callback method is passed a single argument, a `DOMHighResTimeStamp`, which indicates the current time (based on the number of milliseconds since time origin). When



https://developer.mozilla.org/en-US/docs/Web/API/Window/requestAnimationFrame

Capabilities
Web Updates (2020)
Web Updates (2019)
All Articles
December
November
October
September
August
July
June
May
April
March
February
Web Dev Ecosystem team - February wrap up
Exploring a back/forward cache for Chrome
Trust is Good, Observation is Better—Intersection Observer v2
Get Ready for Priority Hints
Replacing a hot path in your app's JavaScript with WebAssembly
Constructable Stylesheets: seamless reusable styles

Intersection Observer v1 is one of those APIs that's probably universally loved, and, now that Safari supports it as well, it's also finally universally usable in all major browsers. For a quick refresher of the API, I recommend watching Surma's Supercharged Microtip on Intersection Observer v1—also embedded below for your viewing pleasure—or reading Surma's in-depth article. People have used Intersection Observer v1 for a wide range of use cases like lazy loading of images and videos, being notified when elements reach `position: sticky`, fire analytics events, and many more.



For the full details, check out the Intersection Observer docs on MDN, but as a short reminder, this is what the Intersection Observer v1 API looks like in the most basic case:

Contents
What's challenging with Intersection Observer v1?
Why is actual visibility such a big deal?
How does Intersection Observer v2 fix this?
What does the new code look like in practice?
Related Links
Acknowledgemen...

https://developers.google.com/web/updates/2019/02/intersectionobserver-v2?hl=en

TTI goals recap...

- Lean harder on native browser features to get you farther along with less (or no!) JavaScript
- Break apart scripts to load only what you need when you need it
- Optimize how the scripts you must run are running in the browser so they're gentler on the CPU